

Paradigmi di Programmazione

Diego Stefanini — prof.ssa Chiara Bodei, a.a. 2026-27

Indice

1	Il lambda calcolo	3
1.1	Calcolabilità e paradigmi	3
1.2	Sintassi dei λ -termini	5
1.3	Variabili libere e legate	6
1.4	Sostituzione e β -riduzione	8
1.5	Ordine di riduzione e strategie	10
2	Programmare nel lambda calcolo	14
2.1	Funzioni con più argomenti	14
2.2	Ricorsione: il combinatore Y	15
2.3	Codifiche: booleani e numeri	16
3	Controllo dei tipi	18
3.1	Perché servono i tipi	18
3.2	Sistemi di tipi	19
3.3	Regole di inferenza	20
3.4	Un linguaggio di espressioni	21

1 Il lambda calcolo

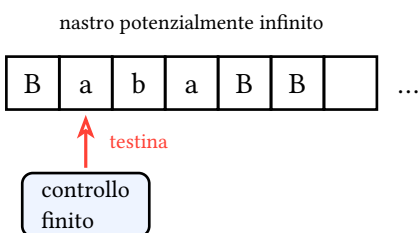
1.1 Calcolabilità e paradigmi

Hilbert, Entscheidungsproblem (problema della decisione): esiste una procedura **del tutto meccanica** che, data una qualunque formula della logica del primo ordine (un'affermazione matematica scritta in un linguaggio formale), dice se è un teorema, cioè se si può dimostrare?

Per rispondere bisogna prima dire cos'è una «procedura meccanica», cioè un **algoritmo**. Tre risposte diverse, quasi insieme:



Com'è fatta la **macchina di Turing**:

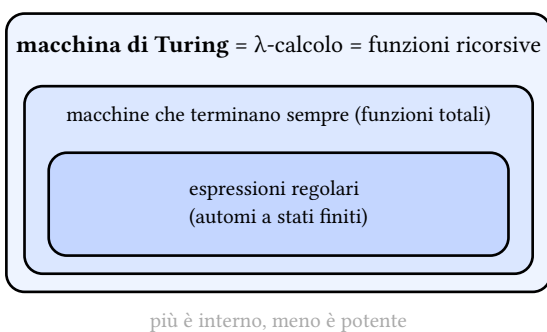


- Macchina **ideale**, non fisica: legge e scrive simboli su un nastro, secondo regole fissate.
- Esempio di regola: *legge a, riscrive a e si sposta a destra.*
- È il modello astratto di una macchina che esegue algoritmi.

Macchina di Turing Universale (UTM): una macchina di Turing che, data la **descrizione** di un'altra macchina e i suoi dati, la simula. È il modello teorico del computer: il programma è un dato.

Con la UTM Turing risponde **no** all'Entscheidungsproblem: esistono problemi che nessuna macchina di calcolo può risolvere.

I tre formalismi, anche se sono fatti in modo diversissimo, calcolano esattamente le stesse funzioni. Da qui la **tesi di Church-Turing**. Nella figura, i modelli meno potenti sono quelli che calcolano solo **funzioni totali** (danno sempre un risultato, per ogni input) e, ancora più dentro, le espressioni regolari:

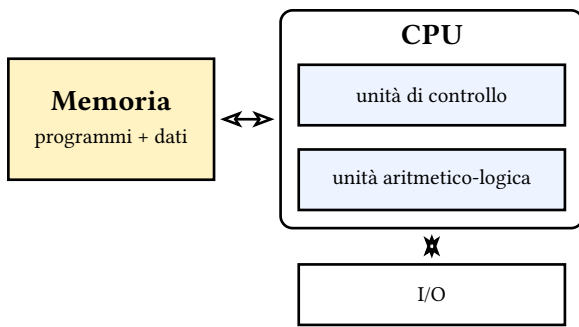


Se una funzione è calcolabile con un qualunque formalismo, allora esiste una macchina di Turing che la calcola.

È **indimostrabile** (i formalismi possibili sono infiniti) ma universalmente accettata: non si conosce nessun modello più potente.

Un linguaggio è **Turing completo** se calcola tutto ciò che calcola una macchina di Turing. Si dimostra scrivendo nel linguaggio un programma che **simula** una macchina di Turing.

Dai due modelli, Turing e Church, nascono due famiglie di linguaggi. Da Turing vengono i linguaggi **imperativi**, passando per la macchina di von Neumann:



Architettura di von Neumann, ispirata alla macchina di Turing ma concreta:

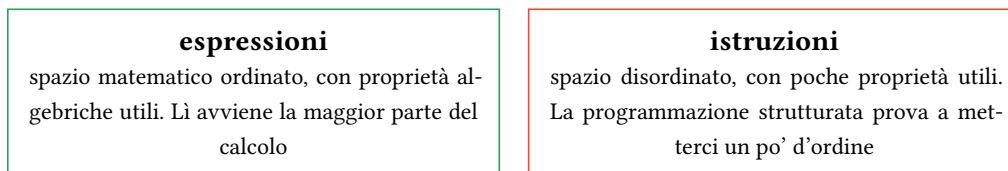
- **memoria** con programmi e dati;
- **CPU** che preleva un'istruzione alla volta, la interpreta e la esegue.

La generalità viene dal **programma memorizzato**: il programma sta in memoria come un dato qualsiasi, quindi la stessa macchina esegue qualunque programma basta caricarlo.

Un linguaggio «alla von Neumann» riproduce ad alto livello questa struttura:

Nel linguaggio		Nella macchina
variabili	↔	celle di memoria (il nastro)
istruzioni di controllo (if, cicli)	↔	test & jump (controllo una condizione e salto)
assegnamento	↔	modifica dello stato (leggo e scrivo la memoria)

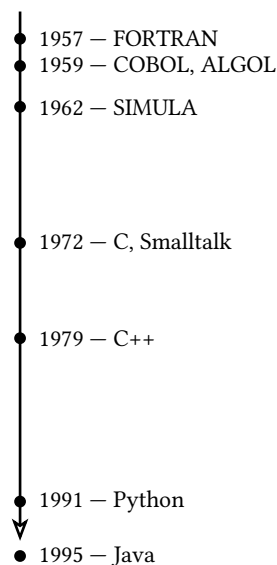
Backus: l'assegnamento divide la programmazione in due mondi.



Da Church vengono invece i linguaggi **funzionali**.

Programmazione funzionale: il programma è una serie di **valutazioni di funzioni matematiche**. Punto di forza: niente **effetti collaterali**. Una funzione ha un effetto collaterale quando, oltre a restituire un valore, cambia qualcosa fuori da sé (una variabile globale, un file, lo schermo). Senza effetti collaterali una funzione con lo stesso input dà sempre lo stesso output, quindi è più facile verificare che il programma sia corretto e ottimizzarlo. Il λ -calcolo (Church, 1935) è il primo linguaggio funzionale.

linguaggi «di Turing»



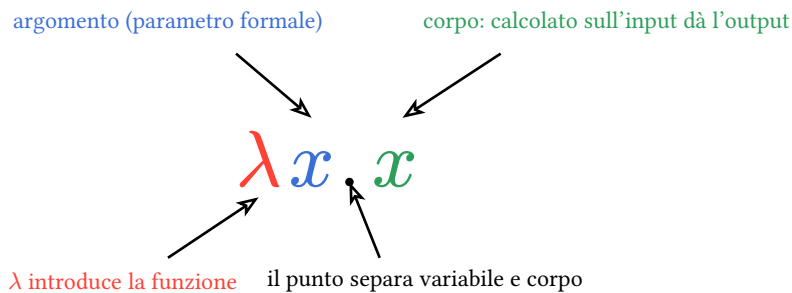
linguaggi «di Church»



Le due linee di discendenza

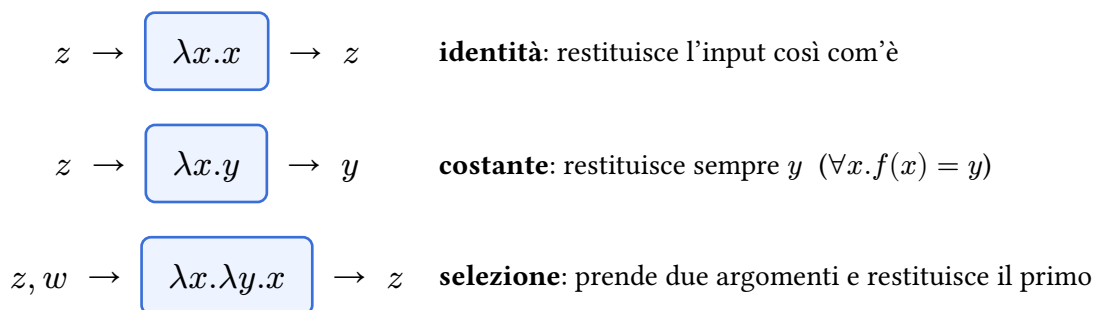
1.2 Sintassi dei λ -termini

Il λ -calcolo ha solo due operazioni. La prima è l'**astrazione**, cioè definire una funzione:



La **legge di corrispondenza** dice cosa fa la funzione su ogni input: qui $\forall x. f(x) = x$, cioè a ogni x associa x stesso. È la **funzione identità**.

La seconda è l'**applicazione**. Applicare una funzione = darle un **parametro attuale** al posto del parametro formale. Immagina la funzione come una scatola:



La selezione passo per passo: $((\lambda x.\lambda y.x)z)w \rightarrow (\lambda y.z)w \rightarrow z$. Nel primo passo metto z al posto di x nel corpo $\lambda y.x$; nel secondo metto w al posto di y nel corpo z , ma y non c'è, quindi w sparisce e resta z .

Queste due operazioni, più le variabili, sono tutto il linguaggio. Un programma è un'espressione (**λ -espressione**). Ci sono solo tre modi di costruirla:

$e ::= x$ variabile

| $\lambda x.e$ astrazione funzionale (dichiarazione di funzione)

| $e e$ applicazione (chiamata di funzione)

Si legge: un'espressione e è una variabile, **oppure** ($|$) un'astrazione, **oppure** un'applicazione. La definizione è **ricorsiva**: dentro $\lambda x.e$ e dentro $e e$ ci sono altre espressioni, costruite con le stesse tre regole. Per esempio $(\lambda x.x)y$ è un'applicazione di $\lambda x.x$ (astrazione, con corpo la variabile x) alla variabile y .

$\lambda x.e$ è una **funzione anonima**: non ha nome, e x è la dichiarazione del suo parametro. Lo stesso concetto nei linguaggi:

Linguaggio	$x \mapsto x + 1$
JavaScript	<code>function(a){ return a + 1; }</code> oppure <code>a => a+1</code>
OCaml	<code>fun x -> x+1</code>
Java (si chiamano lambda)	<code>(int x) -> x + 1</code>

Da qui in poi useremo lettere come e, e_1, e_2, e_3 . **Non** sono variabili del λ -calcolo: sono nomi che stanno per un'espressione qualsiasi, come in algebra $a + b$ vale per due numeri qualsiasi.

In $e_1 e_2$, e_1 è la funzione che chiamo ed e_2 l'argomento che le passo (in JavaScript `e1(e2)`). La funzione non deve per forza avere un nome: la sua definizione può stare **direttamente dentro** la chiamata.

$$\underbrace{(\lambda x. (\lambda y. xy))}_{e_1 = \text{la funzione}} \quad \underbrace{(\lambda z. z)}_{e_2 = \text{l'argomento}}$$

Senza parentesi $\lambda x. xy$ è ambiguo: è $\lambda x. (xy)$ (prende x e restituisce x applicata a y) oppure $(\lambda x. x)y$ (l'identità applicata a y)? Decidono due convenzioni (importanti), che servono anche a scrivere meno parentesi:

1. Lo scope di λ va il più a destra possibile

Il corpo di una λ è **tutto** quello che c'è dopo il punto, fino alla fine (o fino a una parentesi chiusa).

$\lambda x. xy$ è $\lambda x. (xy)$

$\lambda x. \lambda y. xy$ è $\lambda x. (\lambda y. (xy))$

2. L'applicazione associa a sinistra

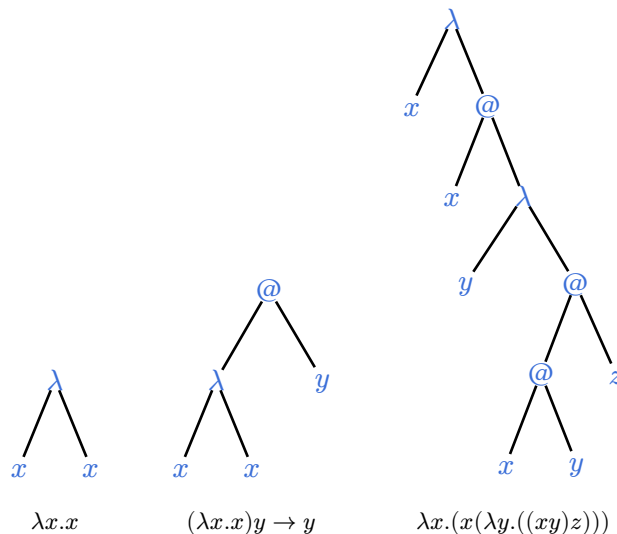
Con più applicazioni di fila si parte da sinistra: prima e_1 applicata a e_2 , poi il risultato applicato a e_3 . Come `f(a)(b)` in JavaScript.

$e_1 e_2 e_3$ è $(e_1 e_2) e_3$

Esercizio: quali parentesi sono sottintese in $\lambda x. x \lambda y. xyz$?

- Regola 1. Il corpo di λx è tutto il resto, $x \lambda y. xyz$; dentro, il corpo di λy è xyz : $\lambda x. (x \lambda y. (xyz))$
- Regola 2. xyz diventa $(xy)z$; e $x \lambda y. \dots$ è x applicata alla funzione $\lambda y. \dots$: $\lambda x. (x (\lambda y. ((xy)z)))$

Ogni termine si può disegnare come un **albero**, che segue l'annidamento completo delle parentesi. Serve a vedere quali passi di valutazione si possono fare. Nodo λ : figlio sinistro il parametro, destro il corpo. Nodo $@$ (applicazione): figli funzione e argomento.



1.3 Variabili libere e legate

λ è un **operatore di binding**: in $\lambda x. e$ lega la x dentro e (il suo **scope**), come $\forall x$ in $\forall x. P$ nella logica. È l'**unico** modo di associare valori a variabili nel λ -calcolo.

- Legata**: introdotta da un λ che la contiene. **verde**, la freccia punta al suo λ .
- Libera**: nessun λ la dichiara. **rossa**.

Espressione	Chi è legato
$\lambda x . x$	x legata
$\lambda x . \lambda y . (x y z)$	x e y legate, z libera
$(\lambda f . f x) y$	f legata, x e y libere
$(\lambda f . f x) f$	la prima f è legata; x e la seconda f sono libere : sta fuori dalle parentesi, fuori dallo scope di λf

Una stessa variabile può voler dire cose diverse in punti diversi:

$$\lambda x . x (\lambda x . x) x$$

che con le parentesi esplicite è $\lambda x . (x (\lambda x . x) x)$

La x nel λx interno è legata a **quel** λ , non a quello esterno. Tre x , due significati: da qui nasce il conflitto di nomi.

Applicando senza fare attenzione ai nomi nasce un problema:

	Espressione	Risultato
ingenuo	$(\lambda x . \lambda y . xy) y$	$\lambda y . yy$ ✗ sbagliato
rinomino y in z	$(\lambda x . \lambda z . xz) y$	$\lambda z . yz$ ✓

Le due espressioni di partenza sono **la stessa funzione** (ho solo cambiato il nome del parametro), ma i risultati si comportano in modo diverso. Nel primo caso la y libera che passo come argomento finisce **catturata** dal λy interno: è un **conflitto di nomi**.

x in $\lambda x . e$ è un segnaposto: si può rinominare con una variabile **fresca**, cioè un nome che non compare da nessuna parte nelle espressioni che stiamo trattando. È così che si risolve il conflitto di nomi, come nella riga «rinomino» della tabella.

Le variabili libere si possono definire in modo preciso, con una **definizione induttiva**: un caso per ogni elemento della sintassi.

$FV(x) = \{x\}$	variabile
$FV(e_1 e_2) = FV(e_1) \cup FV(e_2)$	applicazione: le libere di tutte e due
$FV(\lambda x . e) = FV(e) \setminus \{x\}$	astrazione: il λ lega x , la tolgo

FV sta per *Free Variables*. Una variabile che non è libera è **legata** (*bound*). Un termine senza variabili libere ($FV(e) = \emptyset$) si dice **chiuso**; i termini chiusi si chiamano **combinatori**, il più semplice è l'identità $\lambda x . x$.

Esercizio: $FV((\lambda x . \lambda y . xy)((\lambda z . z)k))$

$$= FV(\lambda x . \lambda y . xy) \cup FV((\lambda z . z)k) = \emptyset \cup \{k\} = \{k\}$$

Cambiare nome a un parametro non cambia niente: $\lambda a . ac$ e $\lambda b . bc$ si dicono **α -equivalenti**, perché a e b contano solo per il **ruolo** che hanno nell'espressione. Lo stesso vale nei linguaggi:

`function(a){ return a + 1; }` è α -equivalente a `function(b){ return b + 1; }`

Espressioni α -equivalenti rappresentano **lo stesso programma**. Rinominare una variabile legata con una variabile fresca si chiama **α -conversione**. Serve a passare da $\lambda x.x$ a $\lambda z.z$, e soprattutto a togliere il conflitto di nomi visto sopra (*variable shadowing*):

$$\lambda \mathbf{x} . \mathbf{x} (\lambda \mathbf{x} . \mathbf{x}) \mathbf{x} \xrightarrow{\alpha} \lambda \mathbf{x} . \mathbf{x} (\lambda \mathbf{z} . \mathbf{z}) \mathbf{x}$$

i due termini sono equivalenti a meno di α -conversione, ma nel secondo ogni nome ha un solo significato

1.4 Sostituzione e β -riduzione

Cosa vuol dire **eseguire** (valutare) una λ -espressione? La valutazione, *eval*, fa solo una cosa: **chiamare funzioni**.

$\text{eval}((\lambda x.e_1)e_2)$: rimpiazzo ogni occorrenza di x in e_1 con e_2 , poi valuto il termine che ne risulta.

La **sostituzione** si scrive $e_1\{x := e_2\}$ (oppure $e_1\{e_2/x\}$): è e_1 con ogni occorrenza **libera** di x sostituita da e_2 . Per esempio $xz\{x := \lambda y.y\} = (\lambda y.y)z$.

Il problema: e se e_2 contiene una variabile libera che in e_1 è legata? Sostituendo alla cieca finisce **catturata** dal λ di e_1 , e legare una variabile libera cambia il significato. Esempio con le operazioni aritmetiche, per semplicità: $(\lambda x.(x * y))\{y := (x + x)\}$.

Come	Risultato
alla cieca	$\lambda x.(x * (x + x))$ ✗ le x di $x + x$ ora sono il parametro
α -conversione, z fresca	$(\lambda z.(z * y))\{y := (x + x)\} = \lambda z.(z * (x + x))$ ✓

La soluzione è la **sostituzione che evita la cattura** (*capture-avoiding substitution*): prima di sostituire, se serve, rinomino con l' α -conversione. La definizione ha un caso per ogni elemento della sintassi:

	Regola	Perché
variabile	$x\{x := e\} \equiv e$	è proprio la variabile da sostituire
	$y\{x := e\} \equiv y$ se $x \neq y$	un'altra variabile non si tocca
applicazione	$(e_1 e_2)\{x := e\} \equiv (e_1\{x := e\})(e_2\{x := e\})$	sostituisco da tutte e due le parti
astrazione	$(\lambda x.e_1)\{x := e\} \equiv \lambda x.e_1$	qui x è legata: la sostituzione vale solo per le variabili libere, quindi non cambia niente
	$(\lambda y.e_1)\{x := e\} \equiv \lambda y.(e_1\{x := e\})$ se $y \neq x$ e $y \notin \text{FV}(e)$	in e non compare y : nessun rischio di conflitto
	$(\lambda y.e_1)\{x := e\} \equiv \lambda z.((e_1\{y := z\})\{x := e\})$ se $y \neq x$ e $y \in \text{FV}(e)$, z fresca	la y libera di e verrebbe catturata: prima α -convertito y in z , poi sostituisco

«Fresca» qui vuol dire che non compare né in e_1 né in e . L'ultimo caso, disegnato: sostituire x con y nel corpo di $\lambda x.\lambda y.xy$.

$\lambda \mathbf{y} . \mathbf{y} \mathbf{y}$

$\lambda \mathbf{z} . \mathbf{y} \mathbf{z}$

alla cieca: la y arriva in un mondo dove esiste
già una y legata e diventa indistinguibile da lei ✗

prima rinomino y in z :
ora la y resta libera ✓

Un esempio completo: $(\lambda x. \lambda y. ((\lambda z. z)x))y$. Il passo $(\lambda y. ((\lambda z. z)x))\{x := y\}$ fatto alla cieca **non va bene**: le due y sono diverse ($y \in \text{FV}(e)$). Quindi, con k fresca:

$$(\lambda y. ((\lambda z. z)x))\{x := y\} \equiv_{\alpha} (\lambda k. ((\lambda z. z)x))\{x := y\} = \lambda k. ((\lambda z. z)y)$$

Esercizi. L'espressione è sempre la stessa, cambia la variabile da sostituire ($\equiv_{\alpha}$ vuol dire « α -equivalente», cioè uguale a meno di rinominare):

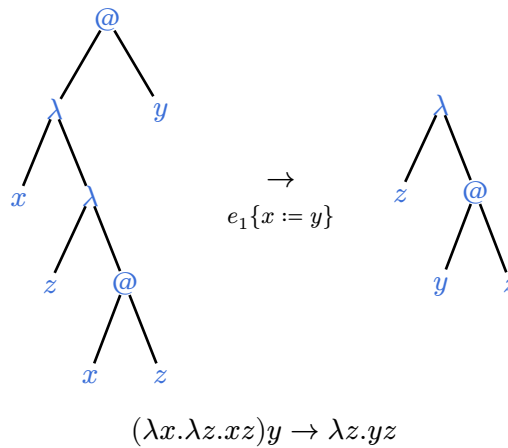
Sostituzione	Risultato
$((\lambda x. yx)w)\{x := \lambda k. kx\}$	$(\lambda x. yx)w$ l'unica x è legata: $(\lambda x. e_1)\{x := e\} \equiv \lambda x. e_1$
$((\lambda x. yx)w)\{y := \lambda k. kx\}$	$\equiv_{\alpha} ((\lambda z. yz)w)\{y := \lambda k. kx\} = (\lambda z. (\lambda k. kx)z)w$ la x di $\lambda k. kx$ è libera e verrebbe catturata da λx : rinomino
$((\lambda x. yx)w)\{w := \lambda k. kx\}$	$(\lambda x. yx)(\lambda k. kx)$ w non è sotto nessun λ : caso applicazione

Con la sostituzione si scrive la regola fondamentale del λ -calcolo, la **β -riduzione**:

$$(\lambda x. e_1)e_2 \rightarrow e_1\{x := e_2\}$$

- Cattura esattamente l'**applicazione di funzione**: $(\lambda x. x)3 \rightarrow 3$.
- Un **redex** (espressione riducibile) è una sottoespressione della forma $(\lambda x. e_1)e_2$, a cui la regola si può applicare.

Esempio: $(\lambda x. \lambda z. xz)y$. Il parametro formale è x , il corpo è $e_1 = \lambda z. xz$, il parametro attuale è $e_2 = y$:



Il λ -calcolo è di **ordine superiore**: una funzione può prendere funzioni come parametri e restituire funzioni come risultato, in modo naturale.

Espressione	Cosa fa
$\lambda x. x$	identità
$\lambda y. (\lambda x. x)$	scarta l'argomento y e restituisce l'identità: è una funzione costante
$\lambda f. f(\lambda x. x)$	data una funzione f , la invoca sull'identità

Applicazione	Passi
$(\lambda x.x)y$	$\rightarrow y$
$(\lambda x.x)(\lambda y.y)$	$\rightarrow \lambda y.y$ l'identità applicata all'identità
$(\lambda x.xy)z$	$\rightarrow zy$ prende una funzione z e la applica a y
$(\lambda x.xy)(\lambda z.z)$	$\rightarrow (\lambda z.z)y \rightarrow y$ il parametro attuale è l'identità
$(\lambda x.((\lambda y.y)x))z$	$\rightarrow (\lambda y.y)z \rightarrow z$
$(\lambda f.fz)(\lambda x.x)$	$\rightarrow (\lambda x.x)z \rightarrow z$ prende una funzione e la applica a z
$(\lambda x.(xx))(\lambda y.y)$	$\rightarrow (\lambda y.y)(\lambda y.y) \rightarrow \lambda y.y$ due termini uguali con ruoli diversi: funzione e argomento
$((\lambda x.\lambda y.x + y)3)5$	$\rightarrow (\lambda y.3 + y)5 \rightarrow 3 + 5 \rightarrow 8$ (supponendo di avere il +)
$((\lambda x.\lambda y.xy)(\lambda x.x))z$	$\rightarrow (\lambda y.(\lambda x.x)y)z \rightarrow (\lambda x.x)z \rightarrow z$ forma $e_1 e_2 e_3$
$(\lambda x.\lambda y.xy)zk$	$\rightarrow (\lambda y.zy)k \rightarrow zk$

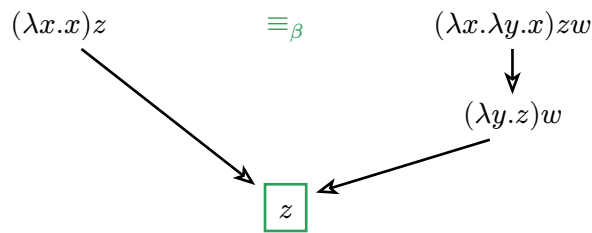
La valutazione va avanti scegliendo un redex e riducendolo. Quando non ci sono più redex l'espressione è in **forma normale β** : non si può più riscrivere con la β -riduzione, ed è il **risultato finale**, il **valore calcolato**. Per esempio $\lambda x.x$ e $\lambda t.\lambda f.t$ sono valori: **le funzioni sono valori**.

Notazione	Significato
$e_1 \rightarrow e_2$	e_2 si ottiene da e_1 con un passo di riduzione
$e_1 \Rightarrow e_2$	e_2 si ottiene da e_1 con zero o più passi

Un passo di β -riduzione è un passo di calcolo, quindi $\Rightarrow$ (la **chiusura riflessiva e transitiva** di $\rightarrow$) rappresenta una computazione qualsiasi. Quando $e_1 \Rightarrow e_2$ si dice che e_1 è **β -riducibile** a e_2 .

Con $\Rightarrow$ si definisce quando due espressioni sono «uguali». e_1 ed e_2 sono **β -equivalenti**, $e_1 \equiv_\beta e_2$, se:

1. sono identiche a meno di α -conversione, oppure
2. $e_1 \Rightarrow e_2$ oppure $e_2 \Rightarrow e_1$, oppure
3. $e_1 \Rightarrow e$ e anche $e_2 \Rightarrow e$ (arrivano alla stessa espressione).

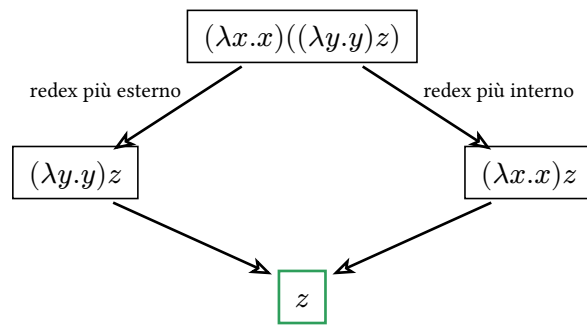


β -equivalenti per il terzo caso: tutte e due si riducono a z

Intuizione: due espressioni sono β -equivalenti quando sono indistinguibili dal punto di vista del calcolo, cioè calcolano gli stessi risultati.

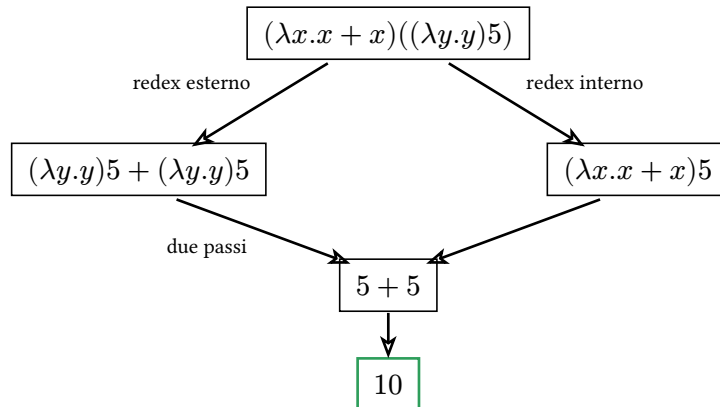
1.5 Ordine di riduzione e strategie

Quando un'espressione contiene più redex, si può scegliere da quale cominciare. In $(\lambda x.x)((\lambda y.y)z)$ ce ne sono due:



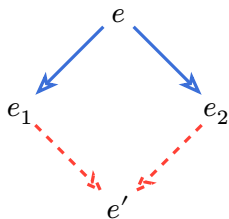
Strade diverse, stesso risultato

Ma in generale l'ordine di valutazione può cambiare il risultato finale? Un altro esempio (usando il +):



Scegliendo prima il redex esterno si copia $(\lambda y.y)5$ e lo si riduce due volte; prima quello interno, una volta sola. Il risultato è lo stesso

La risposta generale è il **teorema di Church-Rosser**: l'ordine in cui si scelgono le β -riduzioni **non influisce sul risultato finale**. Più precisamente: se a una stessa espressione si possono applicare due riduzioni (o sequenze di riduzioni) diverse, esiste un'espressione raggiungibile da entrambi i risultati con altre riduzioni (anche nessuna).



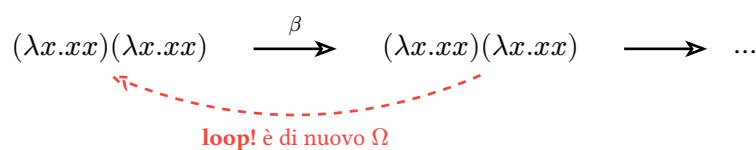
Per questo nell'esempio sopra entrambe le strade arrivano a 10.

Proprietà di confluenza
(o del diamante)

Non sempre però si arriva a una forma normale. Il combinatore

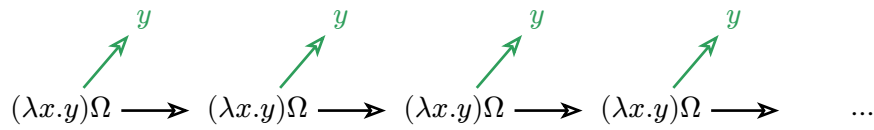
$$\Omega = (\lambda x.xx)(\lambda x.xx)$$

contiene un solo redex, e ridurlo dà di nuovo Ω : la funzione $\lambda x.xx$ applica il suo argomento a se stesso, e l'argomento è proprio $\lambda x.xx$.



Ω non si può ridurre in forma normale: è un combinatore **divergente**, l'equivalente di un ciclo infinito.

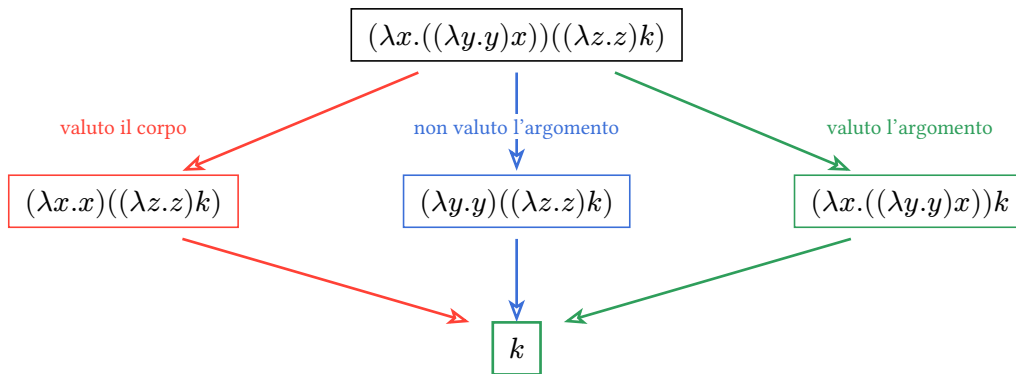
Una stessa espressione può **terminare** facendo certe scelte di riduzione e **non terminare** facendone altre. Per esempio $(\lambda x.y)\Omega$:



In orizzontale riduco dentro Ω e resto sempre fermo; in qualunque momento posso invece applicare $\lambda x.y$, che butta via l'argomento e dà y

Church-Rosser garantisce che **in tutti i casi in cui la riduzione termina, il risultato è lo stesso**: non possono esserci risultati diversi.

Le scelte possibili sono di tre tipi. In $(\lambda x.((\lambda y.y)x))((\lambda z.z)k)$:



Tre scelte diverse, stesso risultato k

Succede lo stesso in un linguaggio. Con `function f(x) { 3+2 + x }` e la chiamata `f(2+1)` posso calcolare l'argomento prima di passarlo (`f(3)`), passarlo non calcolato e lasciarlo alla funzione (`f(2+1)`), oppure semplificare il corpo (`5 + x`).

Una **strategia di valutazione** fissa quale scelta fare. Valutare un'applicazione $(\lambda x.e)e'$ ha tre passi: valuto l'espressione che definisce la funzione, passo il parametro, valuto il corpo. Le due strategie cambiano il secondo passo, il **passaggio dei parametri**:

	Call-by-value (CBV), <i>eager</i>	Call-by-name (CBN), <i>lazy</i>
Cosa fa con l'argomento e'	lo valuta prima della β -riduzione	fa subito la β -riduzione
Al posto di x finisce	il valore dell'argomento	l' espressione non valutata
Redex scelto	il più interno	il più esterno
Linguaggi	OCaml e la maggior parte	Haskell

Le strategie si scrivono con **regole**: sopra la riga l'ipotesi, sotto la conclusione. La prima a sinistra dice: se e_1 fa un passo e diventa e' , allora $e_1 e_2$ diventa $e' e_2$.

Standard (senza strategia) $(\lambda x.e_1)e_2 \rightarrow e_1\{x := e_2\}$ $\frac{e_1 \rightarrow e'}{e_1 e_2 \rightarrow e' e_2} \quad \frac{e_2 \rightarrow e'}{e_1 e_2 \rightarrow e_1 e'}$ $\frac{e \rightarrow e'}{\lambda x.e \rightarrow \lambda x.e'}$	Call-by-value $(\lambda x.e_1)v_2 \rightarrow e_1\{x := v_2\}$ $\frac{e_1 \rightarrow e'}{e_1 e_2 \rightarrow e' e_2}$ $\frac{e_2 \rightarrow e'}{v_1 e_2 \rightarrow v_1 e'}$	Call-by-name $(\lambda x.e_1)e_2 \rightarrow e_1\{x := e_2\}$ $\frac{e_1 \rightarrow e'}{e_1 e_2 \rightarrow e' e_2}$
---	--	---

- **Standard:** qualsiasi redex si può ridurre in qualsiasi momento, anche dentro il corpo di una funzione (terza riga). Non è deterministica.
- **Call-by-value:** v sta per un **valore**, cioè un'espressione già valutata. La β -riduzione scatta solo quando l'argomento è un valore v_2 ; l'argomento e_2 si riduce solo quando la funzione è già un valore v_1 .
- **Call-by-name:** si riduce la parte funzione finché diventa $\lambda x.e_1$, poi si applica. L'argomento non si tocca.

CBV e CBN **non hanno la regola che riduce il corpo di una funzione**. Quindi $\lambda x.((\lambda y.y)z)$ con la riduzione standard diventa $\lambda x.z$, con le due strategie resta così com'è: per loro una funzione è già un valore, anche se dentro ha un redex. È quello che succede nei linguaggi: il corpo di una funzione non viene eseguito finché la funzione non viene chiamata.

La stessa espressione con le due strategie (usando numeri e +):

Call-by-value: redex più interno

$$\begin{aligned} & (\lambda x.\lambda y.yx)(5+2)(\lambda x.x+1) \\ \rightarrow & (\lambda x.\lambda y.yx)7(\lambda x.x+1) \\ \rightarrow & (\lambda y.y7)(\lambda x.x+1) \\ \rightarrow & (\lambda x.x+1)7 \\ \rightarrow & 7+1 \rightarrow 8 \end{aligned}$$

Call-by-name: redex più esterno

$$\begin{aligned} & (\lambda x.\lambda y.yx)(5+2)(\lambda x.x+1) \\ \rightarrow & (\lambda y.y(5+2))(\lambda x.x+1) \\ \rightarrow & (\lambda x.x+1)(5+2) \\ \rightarrow & (5+2)+1 \\ \rightarrow & 7+1 \rightarrow 8 \end{aligned}$$

Conta quale strategia si sceglie?

- Se la valutazione termina, **no**: per la confluenza il risultato è lo stesso.
- La CBV valuta ogni argomento **una volta sola**. La CBN lo valuta **solo se serve**, ma può valutarlo più volte: è la strada di sinistra nella figura di $(\lambda x.x+x)((\lambda y.y)5)$.
- La CBN **termina ogni volta che è possibile**; la CBV può non terminare anche quando la CBN termina. In $(\lambda x.y)\Omega$ la CBN applica subito $\lambda x.y$ e dà y ; la CBV deve prima valutare l'argomento Ω e non finisce mai. Non contraddice la confluenza, che dice che una strada verso il termine comune **esiste**, non quale strategia la trova.

2 Programmare nel lambda calcolo

2.1 Funzioni con più argomenti

Nel λ -calcolo ogni funzione prende **un solo argomento**. Una funzione di due argomenti si scrive come **catena** di funzioni di un argomento, come la selezione $\lambda x. \lambda y. x$: la prima prende x e restituisce una funzione che prende y .

$$\lambda x. (\lambda y. xy) \quad \text{da } f : (X \times Y) \rightarrow Z \text{ a } g : X \rightarrow (Y \rightarrow Z)$$

Questa trasformazione si chiama **currying** (dal logico Haskell Curry). g prende un X e restituisce una funzione da Y a Z , e calcola gli stessi risultati di f :

$$(\lambda x. \lambda y. x + y) 10 \rightarrow \lambda y. 10 + y \quad (\lambda y. 10 + y) 5 = 15$$

$\lambda y. 10 + y$ è una funzione a sé: somma 10 a quello che le passi. Dare a una funzione solo il primo argomento si chiama **applicazione parziale**: ottieni una funzione che aspetta gli altri.

L'espressività non cambia, il risultato è lo stesso che avrei con due parametri insieme:

Definizione	Chiamata
$F = \lambda(x, y). e$	$F(e_1, e_2) \rightarrow e\{x := e_1\}\{y := e_2\}$
$F = \lambda x. \lambda y. e$	$(Fe_1)e_2 \rightarrow (\lambda y. e)\{x := e_1\}e_2 \rightarrow e\{x := e_1\}\{y := e_2\}$

In JavaScript, `curry` prende una funzione `f` che vuole due argomenti insieme e restituisce una funzione che li prende uno alla volta (nota le funzioni annidate):

```
function curry(f) {  
  return function(a) {  
    return function(b) {  
      return f(a, b);  
    };  
  };  
}
```

```
function sum(a, b) { return a + b; }  
  
let curriedSum = curry(sum);  
let h = curriedSum(1); // somma sempre 1  
alert( h(2) );          // 3  
alert( curriedSum(1)(2) ); // 3
```

`sum` passa da `Int × Int → Int` a `Int → (Int → Int)`. I vantaggi del currying:

- **uniformità**: ogni funzione ha un solo argomento, non serve una regola a parte per più argomenti;
- **applicazione parziale**: funzioni specializzate (come `h`) senza costrutti in più;
- **minimalità**: astrazione e applicazione bastano per funzioni con qualsiasi numero di argomenti.

Tutto questo si regge sull'**ordine superiore** (*higher order*). Altri due esempi: `Twice` prende una funzione e la applica due volte:

$$\text{Twice} = \lambda f. \lambda x. f(fx)$$

$$\text{Twice}(\lambda y. y + y) 2 \Rightarrow (\lambda y. y + y)((\lambda y. y + y) 2) \Rightarrow (\lambda y. y + y) 4 \Rightarrow 8$$

`Comp` è la **composizione** $f \circ g$: prende due funzioni e ne restituisce una nuova che applica prima g e poi f . Ha tipo $(B \rightarrow C) \times (A \rightarrow B) \rightarrow (A \rightarrow C)$.

$$\text{Comp} = \lambda f. \lambda g. \lambda x. f(gx) \quad x \rightarrow \boxed{g} \rightarrow gx \rightarrow \boxed{f} \rightarrow f(gx)$$

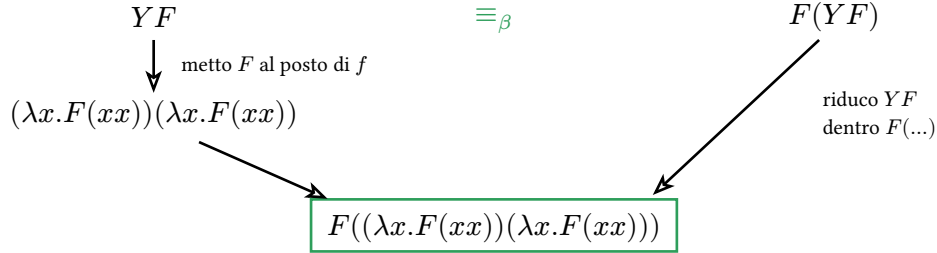
2.2 Ricorsione: il combinatore Y

Nel λ -calcolo non c'è un meccanismo per la ricorsione: le funzioni sono anonime, quindi una funzione non ha un nome con cui richiamare se stessa. Il problema si aggira con il **combinatore di punto fisso** Y .

Un **punto fisso** di una funzione F è un termine p tale che $F(p) = p$. Y è una funzione di ordine superiore che, data F , ne costruisce un punto fisso:

$$Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx)) \quad YF \equiv_{\beta} F(YF)$$

Perché vale la proprietà: YF e $F(YF)$ si riducono allo stesso termine.



Nel secondo passo $\lambda x.F(xx)$ è passata a se stessa come argomento: ogni xx diventa una nuova copia del termine di partenza, con una F davanti. Continuando si ottiene $F(F(...))$: una «copia infinita» della funzione.

Per definire una funzione ricorsiva $F = \langle \text{corpo che contiene } F \rangle$ si fanno due passi:

1. si scrive $G = \lambda f. \langle \text{espressione che contiene } f \rangle$: il nome della funzione diventa un parametro;
2. si definisce $F = YG$.

Allora $F = YG \equiv_{\beta} G(YG)$, cioè l'espressione con YG al posto di f ; e dentro, di nuovo, $YG \equiv_{\beta} G(YG)$. Y «srotola» la definizione una chiamata alla volta, senza usare il nome della funzione. Il caso che ferma la ricorsione lo deve mettere il programmatore.

Esempio: fattoriale (supponendo di avere numeri, if, $n = 0$, $n - 1$, $*$).

$$G = \lambda f. \lambda n. \text{if}(n = 0) \text{ then } 1 \text{ else } n * f(n - 1)$$

Calcolo $YG1$. Per brevità $W = (\lambda x.G(xx))(\lambda x.G(xx))$, il termine che si ricopia:

$$\begin{aligned} YG1 &\rightarrow W1 \\ &\rightarrow GW1 \\ &\rightarrow (\lambda n. \text{if}(n = 0) \text{ then } 1 \text{ else } n * W(n - 1))1 \\ &\rightarrow \text{if}(1 = 0) \text{ then } 1 \text{ else } 1 * W(1 - 1) \\ &\xrightarrow{*} 1 * W0 \\ &\rightarrow 1 * GW0 \\ &\rightarrow 1 * (\lambda n. \text{if}(n = 0) \text{ then } 1 \text{ else } n * W(n - 1))0 \\ &\rightarrow 1 * (\text{if}(0 = 0) \text{ then } 1 \text{ else } 0 * W(0 - 1)) \\ &\xrightarrow{*} 1 * 1 = 1 \end{aligned}$$

Nell'ultimo if la condizione è vera: $W(0 - 1)$ non viene mai calcolato e la ricorsione si ferma.

Y funziona con la **call-by-name**, che non valuta subito gli argomenti: in $F(xx)$ l'argomento xx viene ridotto solo se e quando serve nel corpo di F . Con la call-by-value xx verrebbe valutato subito, e sarebbe un loop infinito.

Per questo in JavaScript, che è call-by-value, Y è un po' diverso. $x(x)$ diventa $y \Rightarrow x(x)(y)$: non è più un valore da calcolare subito ma una funzione che aspetta ancora l'argomento y , e la ricorsione parte solo quando f viene davvero chiamata.

```
const Y = f => (x => x(x))(x => f(y => x(x)(y)));
const factorial = f => (x => (x === 1 ? 1 : x * f(x - 1)));
const YFactorial = Y(factorial)(10);
```

2.3 Codifiche: booleani e numeri

Il λ -calcolo puro ha solo funzioni. Booleani, condizionali e numeri (quello che serve per essere Turing-equivalenti) si ottengono con le **codifiche**. L'idea: non guardare cosa un valore **rappresenta** ma cosa ci si può **fare**, e descrivere quell'uso con una funzione.

Con un **booleano** si fa una scelta fra due alternative. Quindi un booleano è una funzione che, date due scelte, ne seleziona una:

$$\begin{array}{ll} \text{TRUE} = \lambda t. \lambda f. t & \text{TRUE } ab \Rightarrow a \\ \text{FALSE} = \lambda t. \lambda f. f & \text{FALSE } ab \Rightarrow b \end{array}$$

Da qui gli operatori logici: NOT applica il booleano b alle due scelte FALSE e TRUE, così TRUE sceglie FALSE e viceversa.

$$\text{NOT} = \lambda b. b \text{ FALSE TRUE}$$

$$\text{NOT TRUE} \rightarrow \text{TRUE FALSE TRUE} = (\lambda t. \lambda f. t) \text{ FALSE TRUE} \Rightarrow \text{FALSE}$$

$$\text{NOT FALSE} \rightarrow \text{FALSE FALSE TRUE} = (\lambda t. \lambda f. f) \text{ FALSE TRUE} \Rightarrow \text{TRUE}$$

Con la stessa idea, AND e OR usano il primo booleano b per scegliere. AND: se b è vero il risultato è c , altrimenti FALSE. OR: se b è vero il risultato è TRUE, altrimenti c .

$$\text{AND} = \lambda b. \lambda c. b c \text{ FALSE} \quad \text{OR} = \lambda b. \lambda c. b \text{ TRUE } c$$

$$\text{AND TRUE TRUE} \Rightarrow \text{TRUE TRUE FALSE} = (\lambda t. \lambda f. t) \text{ TRUE FALSE}$$

$$\rightarrow (\lambda f. \text{TRUE}) \text{ FALSE} \rightarrow \text{TRUE}$$

$$\text{OR TRUE FALSE} \Rightarrow \text{TRUE TRUE FALSE} = (\lambda t. \lambda f. t) \text{ TRUE FALSE}$$

$$\rightarrow (\lambda f. \text{TRUE}) \text{ FALSE} \rightarrow \text{TRUE}$$

Il **condizionale** sceglie in base al valore della condizione: passa i due rami al booleano c .

$$\text{IF} = \lambda c. \lambda \text{ then}. \lambda \text{ else}. c \text{ then else} \quad \text{IF TRUE } ab \Rightarrow a \quad \text{IF FALSE } ab \Rightarrow b$$

$$\text{IF TRUE } e_1 e_2 = (\lambda c. \lambda \text{ then}. \lambda \text{ else}. c \text{ then else}) \text{ TRUE } e_1 e_2$$

$$\rightarrow (\lambda \text{ then}. \lambda \text{ else}. \text{TRUE then else}) e_1 e_2$$

$$\rightarrow (\lambda \text{ else}. \text{TRUE } e_1 \text{ else}) e_2$$

$$\rightarrow \text{TRUE } e_1 e_2 = (\lambda t. \lambda f. t) e_1 e_2$$

$$\rightarrow (\lambda f. e_1) e_2 \rightarrow e_1$$

Esercizio: calcolare IF FALSE $e_1 e_2$.

I **numeri naturali** hanno una definizione induttiva: zero è un naturale; se n è un naturale, anche Succ n (il successore) lo è. Le operazioni si definiscono allo stesso modo:

	Caso zero	Caso successore
Somma	$m + 0 = m$	$m + \text{Succ } n = \text{Succ}(m + n)$
Prodotto	$m \times 0 = 0$	$m \times \text{Succ } n = m \times n + m$

Nel λ -calcolo un numero si codifica con **cosa ci si fa**: ripetere qualcosa n volte. Il **numerales di Church** C_n prende una funzione s (il successore) e un valore iniziale z (lo zero), e applica s a z per n volte. Il numero 3 vuol dire «fai una cosa tre volte».

$$\begin{array}{lll} C_0 = \lambda s. \lambda z. z & C_1 = \lambda s. \lambda z. sz & C_2 = \lambda s. \lambda z. s(sz) \\ C_3 = \lambda s. \lambda z. s(s(sz)) & C_n = \lambda s. \lambda z. s^n z & \end{array}$$

Le operazioni sui numerali:

Definizione	Come funziona
$\text{SUCC} = \lambda n. \lambda s. \lambda z. s(nsz)$	nsz applica s a z per n volte; poi una s in più: in tutto $n + 1$ volte. $\text{SUCC } C_1 \Rightarrow C_2$
$\text{PLUS} = \lambda m. \lambda n. \lambda s. \lambda z. ms(nsz)$ oppure $\lambda m. \lambda n. m \text{ SUCC } n$	nsz calcola n ; $ms(\dots)$ ci applica s altre m volte. Nella seconda forma: applico SUCC a n per m volte. $\text{PLUS } C_i C_j \Rightarrow \text{SUCC}^i(C_j)$
$\text{TIMES} = \lambda m. \lambda n. m(\text{PLUS } n) C_0$	m conta le ripetizioni: partendo da C_0 , sommo n per m volte. $\text{TIMES } C_i C_j \Rightarrow \text{PLUS } C_j (\text{PLUS } C_j (\dots (\text{PLUS } C_j C_0)))$ con i somme

$\text{SUCC } C_1$ passo per passo. Per non confondere le variabili, in C_1 rinomino s, z in s', z' ; il secondo passo riduce dentro il corpo della funzione (riduzione standard):

$$\begin{aligned} \text{SUCC } C_1 &= (\lambda n. \lambda s. \lambda z. s(nsz)) C_1 \rightarrow \lambda s. \lambda z. s(C_1 sz) \\ &= \lambda s. \lambda z. s((\lambda s'. \lambda z'. s' z') sz) \\ &\Rightarrow \lambda s. \lambda z. s(sz) = C_2 \end{aligned}$$

Esercizi:

- definire ISZERO in modo che $\text{ISZERO } C_0 = \text{TRUE}$ e $\text{ISZERO } C_i = \text{FALSE}$ per $i \neq 0$; calcolare $\text{ISZERO } C_1$;
- calcolare $\text{PLUS } C_1 C_1$.

Quindi il λ -calcolo è **Turing completo**: con codifiche furbe rappresenta quasi ogni costrutto dei linguaggi. Ma i programmi sarebbero lenti (migliaia di chiamate di funzione), lunghi (centinaia di righe) e difficili da capire. In pratica si usano linguaggi più ricchi, che hanno già le primitive giuste (numeri, booleani, `if`).

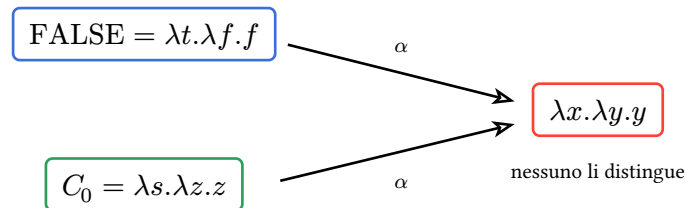
3 Controllo dei tipi

3.1 Perché servono i tipi

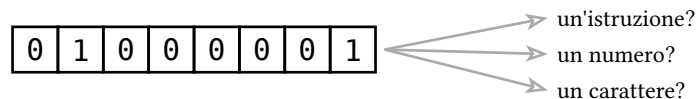
Nel λ -calcolo programmi e valori sono tutti funzioni. Quindi niente impedisce di usare un valore nel posto sbagliato:

$$\text{FALSE } C_0 = (\lambda t. \lambda f. f)(\lambda s. \lambda z. z) \rightarrow \lambda f. f$$

FALSE si può davvero applicare a C_0 , e produce un valore. Ma non ha senso: FALSE serve a scegliere fra due alternative, non a ricevere un numero. Peggio: C_0 e FALSE sono **lo stesso termine**, a meno di α -conversione. Lo stesso vale per $\text{IF } C_0 e_1 e_2$ o $\text{TRUE } C_0$.



Non è solo un problema del λ -calcolo. Nel **linguaggio macchina** le istruzioni sono parole della macchina (sequenze di bit), i dati sono parole della macchina, e le operazioni prendono sequenze di bit e restituiscono sequenze di bit.



Un principio centrale dell'ingegneria del software è **trovare presto gli errori di programmazione** (come $\text{FALSE } C_0$). I linguaggi lo fanno con i **sistemi di tipo** (e più in generale con gli strumenti di **analisi statica**, che esaminano il programma senza eseguirlo).

I **tipi** raggruppano i valori in base al comportamento che vorremmo avessero. Il tipo di un dato dice come il linguaggio permette di usarlo. Un tipo:

- **limita i valori** che un'espressione (una variabile, una funzione) può assumere;
- **definisce le operazioni** ammesse su quei valori e come vengono memorizzati.

I linguaggi hanno dei tipi di base: interi (di varie dimensioni), numeri in virgola mobile (approssimano i reali), caratteri, booleani. $e : T$ si legge « e ha tipo T »:

Espressione	Cosa dice
$3 + 2 : \text{Int} \checkmark$	il risultato è un intero, e $+$ è usato bene: riceve due interi
$\text{true} : \text{Int} \times$	true non è un intero
$2 + \text{true} \times$	$+$ non è applicato bene

Perché controllare i tipi? In un programma grande c'è questo pezzo:

```
if (condizione_complicata)
{ return "hello"/10; }
```

Se i test non rendono mai vera `condizione_complicata`, la divisione di una stringa nel ramo `then` non viene mai scoperta. Poi un input non previsto dai test fa eseguire il ramo, e il programma va in errore **durante l'esecuzione**. Bastava segnalare `"hello"/10` come espressione non ammessa.

Lo stesso codice in JavaScript e in TypeScript:

```
function addNumbers(x, y) {  
  return x + y; }  
console.log(addNumbers(3, "0"));
```

JavaScript stampa "30", una stringa. Il `+` accetta anche una coppia (numero, stringa): il numero viene trasformato in stringa (**coercion**) e le due stringhe attaccate.

```
function addNumbers(x: number,  
                    y: number) {  
  return x + y; }  
console.log(addNumbers(3, "0"));
```

TypeScript si accorge del bug e dà un errore di tipo: *Argument of type "string" is not assignable to parameter of type "number"*. Come? Con delle **regole di tipo**.

3.2 Sistemi di tipi

«*Well-typed programs cannot go wrong*» — Robin Milner

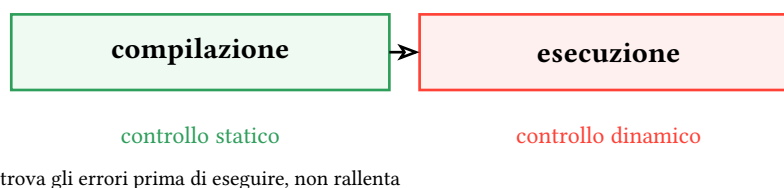
Un **sistema di tipi** associa tipi ai valori calcolati. Seguendo il flusso dei valori, prova a **dimostrare** che non avvengano errori di tipo. È il sistema stesso a stabilire cos'è un errore di tipo: un'operazione che si aspetta un certo tipo di valore usata con un valore per cui non ha senso.

Più precisamente è un metodo **sintattico**, **effettivo** e **strutturale** per dimostrare che il programma non ha comportamenti anomali:

Metodo	Cosa vuol dire
sintattico	l'analisi segue la struttura sintattica del programma
effettivo	c'è un algoritmo che controlla i vincoli sui tipi, da mettere in un compilatore o in un interprete
strutturale	il tipo di un'espressione dipende solo dai tipi delle sue sottoespressioni (composizionale)

Type safety (uso corretto dei tipi): se un'espressione ha un tipo, il tipo garantisce che venga usata nel modo giusto. Senza type safety si scrivono programmi pieni di bug, e i bug sono **vulnerabilità**: un attaccante può sfruttarli per cambiare il comportamento del programma o prenderne il controllo.

Il controllo può essere **statico**, **dinamico** o un misto dei due:



Il **type checker** (controllore dei tipi) verifica che il programma rispetti le intenzioni del programmatore, scritte nelle **annotazioni di tipo** (come `x: number`). Se un programma supera il controllo, durante l'esecuzione **non applicherà mai un'operazione a un valore del tipo sbagliato**. È una **correttezza parziale**: il programma è corretto rispetto ai tipi, non rispetto ad altri errori.

Resta una domanda: come si fa a sapere che i programmi che superano il controllo si comportano davvero bene durante l'esecuzione? Servono due cose:

1. un modo per **specificare** un sistema di tipi;
2. un modo per **dimostrare** che è corretto. Per questo bisogna capire nel dettaglio cosa succede quando un programma viene eseguito, cioè il suo **significato**.

Lo strumento per entrambe sono le regole di inferenza.

3.3 Regole di inferenza

Le abbiamo già usate per le strategie di valutazione: sopra la riga le **premesse**, sotto la **conclusione**, a destra il **nome** della regola. Se valgono tutte le premesse, si può concludere q . Una regola senza premesse è un **assioma**: q è un fatto accettato dal sistema.

$$\frac{p_1 \quad p_2 \quad \dots \quad p_n}{q} \text{ (nome)} \qquad \frac{}{q} \text{ (assioma)}$$

Una conclusione è valida **rispetto alle regole del sistema scelto**. Esempio: le regole per l'uguaglianza fra termini, riflessiva e transitiva:

$$\frac{}{t = t} \text{ (Rifl)} \qquad \frac{t_1 = t_2 \quad t_2 = t_3}{t_1 = t_3} \text{ (Trans)}$$

Una dimostrazione mette insieme più regole in un **albero di derivazione**: le foglie in alto (premesse o assiomi), ogni riga è una regola applicata, la radice in basso è la conclusione. Dalle premesse $a = b, b = c, c = d$ si deriva $a = d$:

$$\frac{\frac{a = b \quad b = c}{a = c} \text{ (Trans)} \quad c = d}{a = d} \text{ (Trans)}$$

Le regole possono anche **definire un insieme**: le regole generano gli elementi, una derivazione dimostra che un elemento ci sta dentro. Per esempio i pari:

$$\{0\} \subseteq \text{Pari} \Rightarrow \{0, 2\} \subseteq \text{Pari} \Rightarrow \{0, 2, 4\} \subseteq \text{Pari} \Rightarrow \dots$$

$$\text{Pari} = \{0, 2, 4, 6, \dots\}$$

$$\frac{}{0 \in \text{Pari}} \text{ (Zero)}$$

Derivazione di $4 \in \text{Pari}$:

$$\frac{(n-2) \in \text{Pari}}{n \in \text{Pari}} \text{ (Passo, } n \geq 2)$$

$$\frac{\frac{}{0 \in \text{Pari}} \text{ (Zero)}}{2 \in \text{Pari}} \text{ (Passo)} \qquad \frac{2 \in \text{Pari}}{4 \in \text{Pari}} \text{ (Passo)}$$

Per dimostrare una proprietà di **tutti** gli elementi generati si segue la struttura delle regole: è l'**induzione sulle derivazioni**. Un caso per ogni regola: gli assiomi sono i casi base, le altre regole i passi induttivi.

Proprietà da dimostrare: $P(n) \equiv n \bmod 2 = 0$.

Caso base: regola (Zero)	Passo induttivo: regola (Passo)
$P(0)$ vale perché $0 \bmod 2 = 0$.	Suppongo $P(n-2)$ e dimostro $P(n)$. $(n-2) \bmod 2 = 0$ vuol dire che esiste k con $n-2 = 2k$. Allora $n = 2k + 2 = 2(k+1)$: con $k' = k+1$ ho $n = 2k'$, quindi $n \bmod 2 = 0$.

Conclusione: per ogni $n \in \text{Pari}$ vale $n \bmod 2 = 0$.

3.4 Un linguaggio di espressioni

Primo caso di studio: un sistema di tipi per un piccolo linguaggio di espressioni con booleani e numeri.

Prima la **sintassi**:

Espressioni	Valori	Valori numerici
$E ::= \text{true}$	$V ::= \text{true}$	$NV ::= 0 \mid 1 \mid 2 \mid \dots$
false	false	
NV	NV	
$\text{if } E \text{ then } E \text{ else } E$		
$\text{succ } E$		
$\text{pred } E$		
$\text{isZero } E$		

I **valori** sono i risultati finali: le espressioni che non vanno più calcolate.

Poi la **semantica**, cioè cosa succede quando un'espressione viene eseguita: un **interprete** scritto con regole di inferenza. Ogni regola dice come fare **un passo** di calcolo $E \rightarrow E'$ (semantica *small-step*, a piccoli passi).

Condizionale

$$\begin{array}{c}
 \text{if true then } E_1 \text{ else } E_2 \rightarrow E_1 \quad (\text{IF-TRUE}) \qquad \text{if false then } E_1 \text{ else } E_2 \rightarrow E_2 \quad (\text{IF-FALSE}) \\
 \\
 \frac{E \rightarrow E'}{\text{if } E \text{ then } E_1 \text{ else } E_2 \rightarrow \text{if } E' \text{ then } E_1 \text{ else } E_2} \quad (\text{IF-COND})
 \end{array}$$

IF-COND ha la stessa forma delle regole delle strategie: finché la condizione non è un valore, si calcola la condizione. Quando diventa true o false scatta IF-TRUE o IF-FALSE.

Operazioni sui numeri

$$\begin{array}{c}
 \frac{E \rightarrow E'}{\text{succ } E \rightarrow \text{succ } E'} \qquad \frac{m = n + 1}{\text{succ } n \rightarrow m} \\
 \\
 \frac{E \rightarrow E'}{\text{pred } E \rightarrow \text{pred } E'} \qquad \text{pred } 0 \rightarrow 0 \qquad \frac{n > 0 \quad m = n - 1}{\text{pred } n \rightarrow m} \\
 \\
 \frac{E \rightarrow E'}{\text{isZero } E \rightarrow \text{isZero } E'} \qquad \text{isZero } 0 \rightarrow \text{true} \qquad \frac{n > 0}{\text{isZero } n \rightarrow \text{false}}
 \end{array}$$

Lo schema è sempre lo stesso: la regola a sinistra calcola l'argomento finché diventa un numero n , le altre fanno l'operazione. $\text{pred } 0$ dà 0: non ci sono numeri negativi.